

EDS Lab Assignments

Practical 1

Name: Sakshi Shivaji Mulay

PRN: 202501040061

1.1.1 Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m \times v$ where:
 m is the mass of the object (in kilograms). v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input()) velocity
= float(input())
momentum= mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays the CODETANTRA online IDE interface. The left sidebar shows the problem title '1.1.1. Calculate Momentum' and a brief description: 'Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula: $p = m \times v$ where: m is the mass of the object (in kilograms). v is the velocity of the object (in meters per second).' It also lists the input and output formats. The main editor area shows the Python code for calculating momentum. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases passed, with a total execution time of 0.045 s. The test cases show expected and actual outputs for mass and velocity values, resulting in the correct momentum calculation.

```
1 #Type Content here...
2 m=float(input())
3 v=float(input())
4 p=m*v
5 print(f"{p:.2f}kgm/s")
```

Test Case	Expected Output	Actual Output
Test case 1	5.0 10.0 50.00kg/s	5.0 10.0 50.00kg/s
Test case 2		

1.1.2

Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places). ☐
- Else print "Invalid"

Code:

```
import math
n=int(input()) sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}") else:
    print("Invalid")
```

Output:

The screenshot displays a coding platform interface for the problem "1.1.2. Conditional Calculation Based on the Number of Digits". The problem statement and constraints are visible on the left. The code editor on the right shows the Python solution. The test results panel at the bottom indicates that 4 out of 4 shown test cases and 3 out of 3 hidden test cases passed. The average time for the test cases is 0.012 s, and the maximum time is 0.015 s. The test case details show the expected output "9" and the actual output "9" for the input "9".

1.1.2. Conditional Calculation Based on the Number of Digits 09:25

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:
The input consists of a single integer n .

Output Format:
If n is a single-digit number, print its square.
If n is a two-digit number, print its square root (rounded to two decimal places).
If n is a three-digit number, print its cube root (rounded to two decimal places).
Else print "Invalid".

Sample Test Cases

condition...

```
1 #write your code here...
2 import math
3 n=int(input())
4 if 1<=n<=9:
5     print(n**2)
6 elif 10<=n<=99:
7     print(f"{math.sqrt(n):.2f}")
8 elif 100<=n<=999:
9     print(f"{math.cbrt(n):.2f}")
10 else:
11     print("Invalid")
```

Average time 0.012 s Maximum time 0.015 s

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 9 ms Debug

Expected output 9 Actual output 9

81 81

Terminal Test cases

< Prev Reset Submit Next >

1.1.3

Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot displays a coding platform interface for the problem '1.1.3. Days between Two Dates'. The left sidebar contains the problem statement, input/output formats, and a note. The main editor shows the Python code. The right sidebar displays test results, including average and maximum execution times, and a table of test cases.

Problem Statement: Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Code:

```
1 from datetime import date
2 from datetime import datetime
3
4 date1 = input().strip()
5 date2 = input().strip()
6
7 d1 = datetime.strptime(date1, "%Y-%m-%d")
8 d2 = datetime.strptime(date2, "%Y-%m-%d")
9
10 difference = d2-d1
11 print(difference.days)
```

Test Results:

- Average time: 0.031 s (30.75 ms)
- Maximum time: 0.064 s (64.00 ms)
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case	Expected output	Actual output
Test case 1	2014-07-02	2014-07-02
	2014-11-02	2014-11-02
	123	123

Reverse a Number

Problem Statement:

1.1.4

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input()) reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot shows the CodeTANTRA online IDE interface. On the left, the problem statement for '1.1.4. Reverse a Number' is displayed, including the input and output formats. The main editor shows a Python program that reads an integer, reverses its digits by converting it to a string and using slicing, and then prints the reversed integer. The program is submitted, and the results panel on the right shows that it passed all test cases. The performance metrics indicate an average time of 0.012 s and a maximum time of 0.014 s. The test cases show expected and actual outputs for two cases, both of which are 5367.

Test Case	Expected Output	Actual Output
Test case 1	5367	5367
Test case 2	7635	7635

Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10. Input Format:

1.1.5

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input()) for
```

```
i in range(1,11):
```

```
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot shows the CodeTANTRA online code editor interface. The title bar indicates the problem is "1.1.5. Multiplication Table". The instructions state: "Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10." The input format is: "The first line of input contains an integer that represents the number for which the multiplication table is to be printed." The output format is: "Print the multiplication table for the given number in the format: <number> x <multiplier> = <result>". A note specifies: "The character 'x' is used in the output to represent multiplication. Refer to the visible test-cases for more clarity." The code editor shows the following Python code:

```
1 #write your code here...
2 n=int(input())
3 for i in range(1,11):
4     print(n, "x", i, "=", n*i)
5
```

The terminal output shows the execution results for test case 1, which passed. The expected output is:

```
8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
```

The actual output is:

```
8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
```

The test case results show that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The average time is 0.014 s and the maximum time is 0.015 s.

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark < 40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)

num_courses = int(input())
marks = list(map(int, input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:", f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

[Home](#)
[Learn Anywhere](#)

202501040061@mitaoe.ac.in
Support
Logout

1.2.1. Pass or Fail
18:35

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

passorFa...
Submit

```

1 # write your code here
2 def cal(marks,total_courses):
3     if any(mark<40 for mark in marks):
4         return "Fail"
5     total_marks = sum(marks)

```

Average time
0.021 s
21.20 ms

Maximum time
0.026 s
26.00 ms

5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed

Test case 1 22 ms Debug

Expected output	Actual output
4	4
55 65 78 89	55 65 78 89
Aggregate Percentage: 71.75	Aggregate Percentage: 71.75
Grade: First-Division	Grade: First-Division

Test case 2 22 ms

Terminal
Test cases

< Prev
Reset
Submit
Next >

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())  
for i in range(n):  
    print(fibonacci(i),end=" ")
```

Output:

The screenshot shows the CODETANTRA online IDE interface. On the left, the problem statement for '1.2.2. Fibonacci Series' is displayed, including the input and output formats. The main editor shows a Python function `def fibonacci(n):` with a recursive implementation. The right sidebar shows the execution results: '2 out of 2 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Below this, two test cases are detailed, showing the expected output '0 1 1 2 3' and the actual output '0 1 1 2 3'. The bottom of the interface has navigation buttons: '< Prev', 'Reset', 'Submit', and 'Next >'.

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("* ",end = "")
    print()
```

Output:

The screenshot shows a coding platform interface with a dark theme. On the left, a sidebar contains the problem title '1.2.3. Pattern - 1', a description 'Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.', input and output format instructions, and a note to refer to displayed test cases. The main area shows a code editor with the following Python code:

```
1 n=int(input())
2 for i in range(1,n+1):
3     print("* "*i)
```

Below the code editor, performance metrics are shown: Average time 0.013 s (13.33 ms) and Maximum time 0.014 s (14.00 ms). Test case results indicate '2 out of 2 shown test case(s) passed' and '4 out of 4 hidden test case(s) passed'. A detailed view of 'Test case 1' shows the expected output as a right-angled triangle of asterisks for input 5, with the actual output matching it. At the bottom, there are buttons for 'Terminal', 'Test cases', 'Start', 'Prev', 'Reset', 'Submit', and 'Next'.

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

The screenshot shows the CodeTANTRA online IDE interface. On the left, the problem description for '1.2.4. Pattern - 2' is visible, including input and output formats. The main editor shows a Python program that takes an integer input and prints a right-angled triangle pattern of numbers. The program has been submitted, and the results show that it passed all test cases. The output for the sample test case (input 5) is displayed as a right-angled triangle of numbers from 1 to 5.

CodeTANTRA Home Learn Anywhere 202501040061@mitaoe.ac.in Support Logout

1.2.4. Pattern - 2 07.55

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

numberP... Submit

```
1 n=int(input())
2 for i in range(1,n+1):
3     for j in range(1,i+1):
4         print(j,end=" ")
5     print()
```

Average time: 0.011 s 11.50 ms Maximum time: 0.014 s 14.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 14 ms Debug

Expected output	Actual output
5	5
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Terminal Test cases

< Prev Reset Submit Next >